

ARCADE

1. Introduction

The Arcade project is a gaming platform that allows for the dynamic loading of graphics libraries and games.

The goal is to provide a modular architecture that makes it easy to add new games and graphical interfaces.

2. General Architecture

The project is divided into three main parts:

2.1 Core

The main program :

- Manages the dynamic loading of libraries
- Display the main menu
- Handle user input
- Manages scores

2.2 Graphics Libraries

Responsible for:

- Displaying sprites and text
- Creating and closing the window
- Retrieving textures, sounds, and text
- Handling events (keyboard, mouse)

Examples :

- ncurses
- SDL2
- SFML

2.3 Games Librairies

Responsible for:

- The game logic
- Manages scores
- Handle key event

Examples :

- Snake
- Demineur

3. Load .so

The project uses the following functions which are in place in the encapsulation folder:

- `dlopen`, `dlsym`, `dlclose`, `dlerror`

3.1 Objectif

Allows:

- Switching graphics libraries
- Dynamically loading different games

This requires encapsulation because these functions originate from C.

4. Interfaces

A global interface for manage the libs :

```
class ILib
{
private:
public:
    virtual ~ILib() = default;
    virtual const std::string get_name() const = 0;
    virtual LibType get_type() const = 0;
};
```

We can therefore distinguish two main types of interfaces

4.1 Graphical Interface (IDisplay)

```
class IDisplay : public ALib {
private:
protected:
public:
    virtual ~IDisplay() = default;
    virtual int createWindow() = 0;
    virtual void closeWindow() = 0;
    virtual void clear() = 0;
    virtual void display() = 0;
    virtual InputEvent getInput() = 0;
    virtual void drawBackground(std::string background_name) = 0;
    virtual void drawText(pos position, float size, std::string text, Color color) = 0;
    virtual void drawSprite(pos position, int rotation, float size, std::string name) = 0;
    virtual int loadTexture(std::string game) = 0;
    virtual int loadSound(std::string game) = 0;
    virtual int playSound(std::string sound_name_in_map) = 0;
    virtual std::string loadName() = 0;
    virtual std::tuple<int, int> recoverWindowSize() = 0;
};
```

4.2 Game Interface (IGame)

```
class IGame : public ALib {
protected:
    std::size_t _score = {0};
    std::vector<Sprite> _sprites;
    std::vector<Texte> _texts;
    std::vector<std::string> _sounds;
public:
    virtual ~IGame() = default;
    virtual void update(InputEvent key) = 0;
    virtual bool isOver() = 0;
    virtual std::vector<Sprite> getSprites() = 0;
    virtual std::vector<Texte> getTextes() = 0;
    virtual std::vector<std::string> getSounds() = 0;
    virtual std::size_t getScore() = 0;
};
```

5. Implement a new Display (Example Ncurses)

5.1 Create the graphic :

The Ncurses graphical must be placed in a dedicated folder :

Graphical/Centipede/

This folder must contain at least two files :

- ncurses.hpp
- ncurses.cpp

6.2 For the centipede.hpp :

Include IDisplay.hpp

- Create a Ncurses class
- Inherit from IDisplay

- Copy ALL methods from IDisplay
- remove virtual
- add override

This should give you:

```
#include "IDisplay.hpp"

class Ncurses : public IDisplay
{
private:
public:
    Ncurses(/* args */);
    int createWindow() override;
    void closeWindow() override;
    void clear() override;
    void display() override;
    InputEvent getInput() override;
    void drawBackground(std::string background_name) override;
    void drawText(pos position, float size, std::string text, Color color) override;
    void drawSprite(pos position, int rotation, float size, std::string name) override;
    std::string loadName() override;
    int loadTexture(std::string game) override;
    int loadSound(std::string game) override;
    int playSound(std::string sound_name_in_map) override;
    std::tuple<int, int> recoverWindowSize() override;
};
```

After that, all you have to do is implement your cpp; don't forget to implement your **EntryPoint** function, otherwise your library will never work.

6.3 For the centipede.cpp :

```
#include "Ncurses.hpp"

Ncurses::Ncurses(/* args */)
{
    _name = "Ncurses";
    _type = LibType::DISPLAY;
}

extern "C" IDisplay *entryPoint()
{
    return new Ncurses();
}
```

Of course, not all methods are implemented; you'll have to do it yourself. You absolutely must use all the methods from the public HPP.

After that, you can do your game.

6. Implement a new Game (Example Centipede)

6.1 Create the game :

The Centipede game must be placed in a dedicated folder:

Games/Centipede/

This folder must contain at least two files :

- `centipede.hpp`
- `centipede.cpp`

6.2 For the `centipede.hpp` :

Include `IGame.hpp`

- Create a Centipede class
- Inherit from IGame
- Copy ALL methods from IGame
- remove `virtual`
- add `override`

This should give you:

```

#include "IGame.hpp"

class Centipede : public IGame
{
protected:
    std::size_t _score = {0};
    std::vector<Sprite> _sprites;
    std::vector<Texte> _texts;
    std::vector<std::string> _sounds;
public:
    Centipede();
    ~Centipede();
    void update(InputEvent key) override;
    bool isOver() override;
    std::vector<Sprite> getSprites() override;
    std::vector<Texte> getTextures() override;
    std::vector<std::string> getSounds() override;
    std::size_t getScore() override;
};

```

After that, all you have to do is implement your cpp; don't forget to implement your **EntryPoint** function, otherwise your library will never work.

6.3 For the centipede.cpp :

```

#include "Centipede.hpp"

Centipede::Centipede(/* args */)
{
    _name = "centipede";
    _type = LibType::GAME;
}

Centipede::~Centipede()
{
}

extern "C" IGame *entryPoint()
{
    return new Centipede();
}

```

Of course, not all methods are implemented; you'll have to do it yourself. You absolutely must use all the methods from the public HPP.

After that, you can do your game.

And your assets in a folder corresponding to its name:

```

assets/centipede/ → ascii → your .txt
                  → images → your .png
                  → sounds your .mp3

```

6.4. Other case :

If you already have .so files with the same architecture as seen above, you just need to place your graphics and game libraries in the folder:

/lib

And your assets in a folder corresponding to its name:

```

assets/centipede/ → ascii → your .txt

```

→ **images** → **your .png**

→ **sounds your .mp3**

7. Authors

Name : Hector Louis, Noam Bouillet

Year : 2026